DON'T GO BROKE IN THE AGE OF AI

After Launch, the Meter Still Runs

[COVER ILLUSTRATION — style-match required]

Where This Booklet Fits: This is booklet 5 of 5 in the Don’t Go Broke series.

Disclaimer: This booklet provides general educational information and is not formal financial, legal, or professional engineering advice.

Table of Contents

Field Note Opening

You built it. You reviewed the draft. You fixed the bugs. You pushed the final button, and now your AI-generated app is live on the internet.

In that moment, there is a profound sense of relief. You survived the blank page, the frustrating prompts, the hallucinated architecture, and the confusing errors. It is natural to feel like you have crossed the finish line.

But building a software product is a project with an end date; operating a software product is a commitment with no scheduled conclusion. Launch is not the end of the work. It is the exact moment when the operational reality—and the recurring costs—begin.

When you were building, you were spending time. Now that the app is live, you are spending money.

Nobody warns you about this part. Every tutorial, every YouTube video, every prompt-engineering guide ends at the same triumphant screenshot: the live URL, the working demo, the "it's done" moment. None of them show you the bill that arrives three weeks later.

That bill is not a bug. It is the direct, predictable cost of a real system doing real work — serving real requests, storing real data, calling real APIs — every second of every day, whether you are watching it or not.

The good news is that this cost is almost always manageable, and often small. The bad news is that "almost always" is not the same as "automatically." Nothing protects you by default. The meters exist whether or not you know where they are.

This booklet is about what happens next. It is about how to survive the first week, how to find the hidden meters that are quietly spinning up costs, and how to operate your live app without letting it bankrupt you.

Section 1
The Invisible Meters

You are no longer paying for a one-time product. You are renting infrastructure by the millisecond. Every time a user clicks a button, a tiny invisible meter ticks up.

There are four main meters that run continuously:

1. **Compute:** The server power required to run your code.
2. **Storage and Logs:** The database space required to save your users' information, and the space required to save system error records.
3. **Third-Party APIs:** External services (like sending emails, processing payments, or fetching weather data) that charge you per event.
4. **AI Tokens:** The cost of asking the AI model to process text or generate an answer for your users.

AI makes building fast and cheap, but it does not make running the app free. If your app becomes popular overnight, those invisible meters will spin faster than you can blink.

Why "It's Just a Small App" Doesn't Protect You

Founders often assume a small, low-traffic app is automatically a cheap one. That assumption fails in a specific, predictable way: cost is not driven by how many users you have. It is driven by how many times your code runs, and what that code is allowed to do each time it runs.

A single user can trigger the same expensive operation a thousand times in an afternoon, whether by accident (a buggy loop retrying itself) or by curiosity (someone testing how far they can push a feature). A meter does not know or care whether the request came from one enthusiastic user or a thousand casual ones. It only counts what happened.

This is why a founder can launch to total silence — no press, no traffic, no signups — and still wake up to a real bill. The meters were never watching for popularity. They were watching for activity, and activity does not require an audience.

Consider two apps launched on the same day. App A gets 500 real users who each use it once, briefly, exactly as intended. App B gets zero real users, but has one

background job silently retrying a failed API call every sixty seconds. App B's bill can easily exceed App A's, because the meters do not measure success. They measure what your code was told to do, and how often it was allowed to do it.

Nobody launches an app hoping this happens. But nobody prevents it either, unless they go looking for it — because the AI that built your app was never asked to think about cost. It was asked to make the feature work, and it did exactly that, without a second thought about what happens if that feature runs ten thousand times instead of ten.

The After-Launch Cost Map Worksheet

Before you can control your costs, you must map them. Use this worksheet to document exactly where your app is spending money.

- ☐ **Hosting Provider:** Where does the code live? (e.g., Vercel, Heroku, AWS)
  - *Billing Tier:* _____
  - *Hard Cap Set At:* $_____
- ☐ **Database Provider:** Where is the user data stored? (e.g., Supabase, MongoDB, Firebase)
  - *Billing Tier:* _____
  - *Hard Cap Set At:* $_____
- ☐ **AI Provider:** Who provides the intelligence? (e.g., OpenAI, Anthropic, Gemini)
  - *Billing Tier:* _____
  - *Hard Cap Set At:* $_____
- ☐ **Email/SMS Provider:** Who sends the notifications? (e.g., Resend, Twilio)
  - *Billing Tier:* _____
  - *Hard Cap Set At:* $_____

Section 2
Provider-by-Provider Cost Review

You must review each major cost center individually. Use this beginner-safe guidance to check your providers before traffic arrives.

Hosting (Compute)

- **What keeps spending money:** High traffic, unoptimized code that takes too long to run, or serverless functions that don't shut down efficiently.
- **What to check first:** The billing tier and execution limits.
- **What alert to set:** Set a monthly spend alert at 50% and a hard cap at 100% of your budget.

Database

- **What keeps spending money:** Frequent read/write operations and growing storage from unarchived data.
- **What to check first:** Are there operations that query the entire database every time a page loads?
- **What alert to set:** Set an alert on total storage size and read/write operations per day.

Storage (Files & Images)

- **What keeps spending money:** Users uploading massive files, or the system failing to delete files when users delete their accounts.
- **What to check first:** Is there a maximum file size restriction on all upload forms?
- **What alert to set:** Set an alert on total bucket bandwidth and storage volume.

Logs

- **What keeps spending money:** Storing thousands of error messages generated by a single bug.
- **What to check first:** Is there a log retention policy in place?
- **What alert to set:** Set a cap so logs older than 7 or 14 days are automatically deleted.

AI / API Credits

- **What keeps spending money:** Uncapped text generation and users clicking "Submit" repeatedly.
- **What to check first:** Is there a max-token limit on every AI request?
- **What alert to set:** Set a hard billing limit directly in the AI provider's dashboard.

Email Sending

- **What keeps spending money:** Spam bots triggering password resets or welcome emails thousands of times.
- **What to check first:** Does the signup form have bot protection?
- **What alert to set:** Set a daily sending limit well below your monthly budget.

Authentication / Account Services

- **What keeps spending money:** High volumes of monthly active users (MAUs) or SMS-based multifactor authentication.
- **What to check first:** The cost-per-user on your identity provider's free tier.
- **What alert to set:** Set an alert for MAU growth thresholds.

Monitoring / Analytics

- **What keeps spending money:** Sending every single user click to a paid analytics platform.
- **What to check first:** Are you tracking only essential events, or tracking everything indiscriminately?
- **What alert to set:** Alert on total event volume per month.

[ILLUSTRATION — style-match required]

Section 3
Background Jobs and Silent Loops

The most dangerous cost in a modern application is the one that happens while nobody is using it.

When you ask an AI to build a complex feature, it will often write a "background job"—a process that runs automatically on a schedule, behind the scenes.

Quiet Cost Leak: A Worked Example

A founder asked an AI to build a feature that pulls new articles from an RSS feed and summarizes them. The AI wrote a background job (a "cron job") to check the feed every 60 seconds.

The founder launched the app on a Friday and went away for the weekend. The app had exactly zero users. But every 60 seconds, the background job woke up, fetched data, called the AI API to attempt a summary, failed due to a missing article, wrote the error to a log file, and went back to sleep.

What Looked Safe: The app was live, the UI was clean, and there were no users clicking buttons. The dashboard looked calm. **Where the Cost Came From:** A silent, scheduled retry loop that triggered a third-party AI call and generated a database write every 60 seconds. **The Result:** By Monday, the app had run 4,320 times. It burned through the founder's AI API credits and filled up the database's storage limit with 4,320 identical error logs. The bill was over $200, all for an app with no users.

The Quiet Cost Leak Diagnostic

You must audit your app for silent loops immediately after launch.

- ☐ **Scheduled Tasks:** Are there any cron jobs running on a schedule? (Every minute, every hour, every day?)
- ☐ **Retry Logic:** If a task fails (like an email failing to send), does it try again forever?
- ☐ **Webhooks:** Is your app waiting for external events (like a payment confirmation) to trigger a database update?

Section 4
Logs, Storage, and Database Review

Compute and AI tokens get all the attention, but boring infrastructure can bankrupt you just as quickly.

The Logging Trap Every time an error happens, modern apps write a "log." If your app has a bug that triggers an error every time a user scrolls, your app might generate 10,000 logs a day. Log storage is not free. If you do not monitor your logs, your database bill will skyrocket just from storing records of your own app's failures.

The File Upload Trap If your app allows users to upload profile pictures or PDFs, you are paying for storage. What happens if a malicious user uploads a 5-gigabyte movie file? Did the AI put a size limit on the upload form? If not, you are paying for their movie storage.

The Email Trap Every password reset email, welcome email, and notification costs a fraction of a cent. If a bot discovers your signup form and creates 10,000 fake accounts, you will pay for 10,000 welcome emails.

The Test Data Trap Every AI-assisted build leaves fingerprints. When you were testing your app, you and the AI likely created dozens of dummy accounts, sample products, and placeholder images to check that the features worked. Most builders forget these exist. That test data still occupies real storage space, gets backed up in every nightly snapshot, and in some cases still counts against your record limits and API quotas exactly the same as a paying customer's real data would.

The Backup Multiplier Most hosting and database providers automatically create nightly backups or snapshots for you, often without asking. This is a good safety practice, but it is not free: a database that holds 2GB of data can quietly cost you 2GB, 4GB, or more every month once backup retention is added on top of the live copy. Check your provider's backup retention settings the same way you check your log retention settings — an unbounded backup history is just another silent meter.

The Orphaned Record Trap When a user deletes their account, does everything they created disappear too? Many AI-generated apps delete the user's login credentials but leave behind their uploaded files, generated content, and log history — orphaned rows nobody queries, and nobody deletes, running up your storage bill indefinitely for someone who is no longer even using your app.

A Quiet Storage Bill: A Worked Example

A builder tested a photo-upload feature by uploading forty sample images at full resolution — never bothering to compress them, since it was "just a test." Six months later, those forty images were still sitting in the production storage bucket, backed up nightly, indexed in searches, and billed at the same per-gigabyte rate as any real user's photo. Nobody had deleted them because nobody remembered they were there. The fix took ten minutes. Finding the problem took six months of quietly rising bills.

None of this requires exotic tooling. A single scheduled cleanup job — one that runs weekly and deletes anything expired, orphaned, or explicitly marked as test data — solves the majority of these leaks permanently. The discipline is not technical. It is remembering that storage is not a filing cabinet you fill once. It is a meter you pay every single month for everything you have ever kept.

Logs / Storage / Database Review Sheet

Use this beginner-safe workflow to review your storage infrastructure:

- ☐ **Check Log Growth:** Are logs growing quickly? Are old error logs automatically deleted after 7 days to save space?
- ☐ **Check Upload Limits:** Are file uploads strictly capped at a safe size (e.g., 5MB), and are files deleted when a user deletes their account?
- ☐ **Check Failed Records:** Are failed records (like unsent emails) piling up in a queue table indefinitely?
- ☐ **Check Test Data:** Was test data left in production, taking up valuable database space?
- ☐ **Check Generated Content:** Are old AI-generated text results being saved forever even if the user abandons them?

[ILLUSTRATION — style-match required]

Section 5
AI Credit Usage After Launch

If you built an app that relies on an AI API (like OpenAI or Anthropic), you face a unique operational risk.

Traditional APIs are cheap and predictable. Large Language Model (LLM) APIs are expensive and highly variable. The cost depends entirely on how much text the user types and how much text the AI generates.

Scenario: The Uncapped Feature

A builder launched an AI writing assistant. The AI charged per token (word). The builder assumed typical users would write a few paragraphs a day.

Instead, a small group of three power users pasted entire 500-page novels into the tool, asking the AI to rewrite them. The builder had not set a character limit on the text input box. Those three users burned through the builder's entire monthly budget in a single afternoon. A "successful" launch with highly engaged users resulted in a devastating bill because the feature was left uncapped.

"Find Uncapped Cost Paths" Prompt

- ☐ **Input Limits:** Is there a strict character limit on every text box that talks to the AI?
- ☐ **Output Limits:** Is the AI's response capped at a specific maximum length (e.g., `max_tokens: 500`)?
- ☐ **Rate Limiting:** Is a user prevented from clicking the "Generate" button 50 times in one minute?

"Audit this codebase for uncapped AI and API usage.

1. List every text input that is sent to an AI API. Does it have a strict character limit?
2. List every API call that is triggered by a button click. Does it have rate limiting (e.g., max 5 clicks per minute)?
3. If any inputs or buttons are uncapped, generate the code to restrict them immediately."

Section 6
The Post-Launch Dashboard Review

You cannot manage costs if you do not look at them. Every major platform (hosting, database, AI provider) has a dashboard. You must learn to read them.

Dashboard Walkthrough

Log in to the dashboard of your primary platform. Look for these tabs:

- **Where to look for usage:** Find the "Usage" or "Analytics" tab. Look for the number of requests, functions executed, or bandwidth used.
- **Where to look for billing:** Find the "Billing" or "Invoices" tab. Check the "Current Estimated Spend."
- **Where to look for errors:** Find the "Logs" or "Errors" tab. A sea of red text means the app is struggling, which often means it is wasting compute power retrying failed actions.
- **Where to look for background activity:** Look for "Cron Jobs," "Workers," or "Scheduled Tasks." Are they running? Are they failing?
- **What "normal" looks like:** A healthy new app should have very low flatline usage, punctuated by small, brief bumps when actual users interact with it.
- **What should trigger a pause:** If the usage graph looks like a steep mountain climbing straight upward while user registrations remain flat, something is caught in a loop. Pause the service immediately.

Reading the Trend Line: A Worked Example

A builder checked her dashboard on day one and saw a small, flat line of usage — a handful of requests, a few database writes, nothing alarming. She checked again a week later and saw the exact same flat line, still low, still calm. Confident that nothing had changed, she stopped checking.

Three weeks after that, a routine notification email arrived: her database had grown to eighteen times its original size. Nothing had crashed. Nothing looked broken from the outside. But a background job that ran on every page load had been quietly duplicating records for weeks, and because she never looked at the dashboard again after that first calm check, nobody noticed until the storage bill tripled.

The lesson is not that she checked wrong. It is that a single glance at a dashboard tells you what is happening right now, not what has been happening since the last time you looked. This is why the weekly cadence in Section 9 exists — not because any single check is unreliable, but because cost problems compound quietly between checks.

What Good Dashboard Habits Look Like

Compare, don't just observe. A number by itself means little. Ten thousand requests a day is either normal or alarming depending entirely on what it was last week. Always look at the trend, not the snapshot.

Read the graph shape, not just the total. A steady staircase climbing in step with new signups is healthy growth. A vertical spike with no matching signup spike is a leak. The shape tells you which one you are looking at before the total dollar figure does.

Trust the alert, not your memory. You will not remember to check every dashboard every day, and you are not supposed to. That is what the budget alerts from Section 7 are for. The dashboard is where you investigate after an alert fires — not something you are expected to babysit around the clock.

Most builders only open these dashboards twice: once out of curiosity right after launch, and once in a panic when a bill arrives. Reading the dashboard is not busywork. It is the only way you find out your app is behaving badly before your bank account does. Bookmark these tabs — the five minutes it takes to check them on a slow Tuesday is far cheaper than the hour it takes to diagnose a runaway bill on a Saturday.

[ILLUSTRATION — style-match required]

Section 7
Usage Spikes and Surprise Bills

The most dangerous assumption a first-time builder can make is, "I'll just check the billing dashboard every few days."

Do not trust your future self to remember. An unexpected traffic spike, a bot caught in a loop, or a poorly optimized database query can rack up hundreds of dollars in costs while you are sleeping.

You must set traps.

The Budget Alert Setup Checklist

Before the first user arrives, go into your hosting and AI provider dashboards and set up strict budgets. Complete this checklist across every provider:

- ☐ **Hosting Budget Alert:** Alert set at $**. Hard limit set at $**.
- ☐ **Database/Storage Alert:** Alert set at $**. Hard limit set at $**.
- ☐ **AI/API Credit Limit:** Alert set at $**. Hard limit set at $**.
- ☐ **Email/Send Limit:** Alert set at ____ emails/day.
- ☐ **Logging Retention Limit:** Logs automatically delete after ____ days.
- ☐ **Payment Provider Alert:** If a payment processor like Stripe fails repeatedly, does it alert me?

Section 8
When the Bill Spikes: A Full Walkthrough

You wake up to an email: "*Action Required: 80% of monthly budget reached.*" It is only the 3rd of the month.

Case Study: Surviving the Spike

The First Reaction to Avoid: Do not panic, and do not immediately log in and raise the credit limit just to keep the site online. If you raise the limit without finding the leak, a $50 problem will become a $500 problem by tomorrow.

What Number Changed: Open the dashboard. Did bandwidth spike? Did AI tokens spike? Did database writes spike?

What Service Caused It: You see that AI tokens spiked massively overnight.

Separate Real Growth from Runaway Automation: Check your user registrations. Did 1,000 new people sign up? If yes, congratulations, you have a scaling challenge. But if you have zero new users and 50,000 AI tokens burned, a bot or a loop is draining your account.

Pause the Expensive Path: Do not take the whole app offline if you don't have to. Go into the code, find the AI generation function, and comment it out or disable the route. Deploy the change. The bleeding stops.

Ask the AI for an Explanation: Now that the bleeding is stopped, copy the code for the expensive feature and paste it into your AI builder. Ask: "*This feature just caused a massive billing spike overnight. What vulnerability allows it to be called repeatedly, and how do I lock it down?*"

Decide: Once the AI identifies the missing rate limit or the infinite loop, apply the fix. Resume the service cautiously and watch the dashboard.

A Second Case: When the Spike Is Good News

Not every spike is an emergency. A different founder got the identical 80%-of-budget alert on the same day her app was featured on a popular newsletter. AI token usage had tripled overnight, exactly like the runaway-bot scenario above.

She followed the same first steps — checked the dashboard, saw AI tokens had spiked — but this time, user registrations had also tripled, matching the spike

almost exactly. There was no mystery to solve. She had real growth, not a runaway loop.

Her response was the opposite of a pause: she raised her spending limit deliberately, watched the dashboard for the rest of the day to confirm the new numbers held steady, and treated the alert as a signal to plan for a higher permanent budget going forward — not as a fire to put out.

The two founders received the identical alert email. One had a leak; one had a launch. The dashboard, not the alert itself, is what told them apart.

What Not to Do: Do not delete your API keys entirely in a panic — you may need them minutes later once you've confirmed the fix. Do not downgrade your hosting tier mid-spike — that can cause outages for real users if the spike turns out to be genuine growth. And do not wait for the bill to arrive before investigating — by the time an invoice lands, the same leak has usually already run for another full billing cycle.

Speed matters more than certainty here. You can always undo a cautious pause. You cannot undo a bill that already ran overnight while you deliberated.

[ILLUSTRATION — style-match required]

Section 9
The Weekly Operating Cadence

Operating an app means establishing a rhythm. Once the launch excitement fades, adopt a strict weekly operating cadence. Pick a day (like Friday afternoon) and run this checklist.

The Weekly Rhythm

- ☐ **Check Spend:** Open the billing tabs for your host, database, and AI provider. Is the estimated bill on track?
- ☐ **Check Usage:** Did bandwidth or API calls suddenly trend upward?
- ☐ **Check Errors:** Open the logs. Are there new recurring errors that need to be fixed before they compound?
- ☐ **Check Logs/Storage Growth:** Are old files actually being deleted, or is storage creeping up?
- ☐ **Check AI/API Calls:** Are the input/output caps still holding, or did a user find a way around them?
- ☐ **Check Email Volume:** Is the send rate normal, or is a bot spamming forms?
- ☐ **Check Recent Changes:** Did you push any code this week that might have introduced a silent loop?
- ☐ **Decide:** What needs to be paused, capped, cleaned, or safely continued next week?

Section 10
Feature-Cost Review Before Adding Anything New

By Week 2, users will start asking for new things. The reality of maintenance begins here.

Builders often make the mistake of adding complex new features before understanding the operating cost of the existing ones. Do not let momentum blind you.

The excitement of a growing user base makes this easy to forget. A feature request from an engaged user feels like validation, and validation makes builders want to ship immediately. That instinct is not wrong — it just needs one gate in front of it before the code goes live.

The Safe Feature Addition Gate

Before you add any new feature after launch, you must force the AI to answer these questions:

- ☐ What external services does this feature call?
- ☐ Does this feature use AI, and if so, is it strictly capped?
- ☐ Does this feature send email, and is it rate-limited?
- ☐ Does this feature store files, and do they expire?
- ☐ Does this feature run automatically in the background?
- ☐ Can a malicious user trigger this feature 100 times in a row?
- ☐ What happens to my database if 10 users use this feature?
- ☐ What happens to my bill if 100 users use this feature?
- ☐ What new alert or cap must I set up to protect this feature?

Skipping the Gate: A Worked Example

A founder's users kept asking for a "summarize my document" button. It seemed simple enough to justify skipping the checklist — after all, it was just one more feature on an app that was already working fine. The AI built it in an afternoon: upload a file, the AI reads it, the AI summarizes it, done.

Nobody asked what happens if someone uploads a 400-page PDF. Nobody capped how many times a single user could click "Summarize." Nobody set an output length limit on the response. Two days after shipping, a single enthusiastic user uploaded eleven long documents back to back, and the feature alone generated more AI token cost than the entire rest of the app had generated in its first month combined.

The fix — a file size cap, an output length limit, and a five-per-hour rate limit — took less time to build than the original feature. The founder simply had not asked the nine questions before shipping.

Why This Feels Slow, and Why It Isn't

Running nine questions past the AI before adding a feature can feel like bureaucracy standing between you and shipping something your users are excited about. It is not. Each question takes seconds to ask and seconds for the AI to answer. The entire gate can be run in the time it takes to make coffee.

Compare that to the alternative: shipping first and finding out the true cost only after a bill arrives, at which point you are not just adding a cap — you are also explaining to yourself why a single feature wiped out a month of revenue. The gate is not a tax on moving fast. It is the fastest path to a feature you never have to revisit in a panic.

Run the gate every time, even for features that feel small. "Small" is a description of how the feature looks in a demo. It says nothing about how many times a real user, or a hundred real users, will click the button once it is live.

If the AI cannot answer these questions clearly, the feature is too dangerous to deploy.

Section 11
Pause, Scale Down, Keep Going, or Shut Off

There is a taboo around killing a project. Many builders will spend fifty dollars a month keeping an abandoned app alive just because they feel guilty about shutting it down.

If the recurring costs outweigh the value the app provides, you have to make a hard decision. Do not let zombie apps slowly drain your bank account.

The Pause / Scale Down / Shut Off Playbook

Use this decision guide to control an app that is no longer behaving exactly as you want:

1. PAUSE (The Emergency Brake) *When to use it:* A bug is burning through your API credits, or a bot is spamming your database. *Action:* Pause a feature, but keep the app live. You can temporarily disable the "Generate AI" button, or temporarily revoke the third-party API keys to stop the bleeding. Fix the code locally, then turn it back on.

2. SCALE DOWN (The Baseline Retreat) *When to use it:* The app is stable, but nobody is using the expensive features, or you can no longer afford the current tier. *Action:* Turn off AI generation entirely but keep the previously saved results visible. Reduce logging retention from 30 days to 7 days. Disable background jobs temporarily. Cap file uploads. Downgrade your database from the $20/month tier to the $5/month tier. Lower the baseline cost.

3. KEEP GOING (The Green Light) *When to use it:* The app is generating enough value (or revenue) to easily cover its run rate. The logs are clean. The background jobs are sleeping appropriately. Budget alerts are active. *Action:* Keep going only after alerts and limits are in place. Proceed to the next phase of the product lifecycle. You have earned the right to start building new features.

4. SHUT OFF (The Graceful Shutdown) *When to use it:* The app costs more to run than it earns. You have lost interest in maintaining it. The codebase has rotted. *Action:* Do not just walk away. Execute a clean shutdown.

The Shutdown / Pause Checklist

If you decide to shut down or pause the app, use this safe checklist to ensure you leave no loose ends:

- ☐ **Preserve Data:** Export critical user data or business records you might need later.
- ☐ **Turn Off Public Access:** Change DNS settings or put up a maintenance page so users know the app is gone.
- ☐ **Disable Background Jobs:** Delete all cron jobs and webhooks immediately.
- ☐ **Disable AI/API Calls:** Revoke your API keys in the third-party provider dashboards so they can never be used again.
- ☐ **Reduce Storage:** Delete all logs, generated files, and user uploads to stop storage billing.
- ☐ **Cancel Services:** Downgrade to free tiers or cancel paid subscriptions completely.
- ☐ **Keep Billing Records:** Save your final invoices for tax or accounting purposes.
- ☐ **Document It:** Write down a brief note on why the project was paused so you don't forget the lesson six months from now.

[ILLUSTRATION — style-match required]

Field Note Closing

Building an app with AI is an incredible achievement. Launching it takes courage. But operating it requires maturity.

By setting hard billing limits, hunting down silent background jobs, and anticipating the true cost of users, you transition from someone who just got lucky with an AI builder to someone who knows how to run a software business.

Look at how far this has come. You started this series with an idea you weren't even sure deserved to become a project. You forced yourself to name the problem before naming the product, find one real user instead of chasing "everyone," and decide what version one would deliberately leave out. Then you turned that idea into a blueprint an AI could actually build from, instead of a vague hope it had to guess at. You learned to hand an agent narrow, bounded tasks instead of the keys to the whole codebase, and to treat its plans with the same scrutiny you would give a new hire on their first day. You learned not to trust a draft just because it looked finished, and to interrogate a "done" feature until you actually understood what it did and did not do. And now, here, you have learned that shipping the app was never the finish line. It was the moment the real, ongoing job began.

Every one of those lessons protects you from the same underlying mistake: letting a fast, confident, tireless tool make decisions on your behalf that you never actually reviewed. The idea stage protected you from building the wrong thing. The blueprint stage protected you from building it on a foundation nobody understood. The agent stage protected you from a tireless assistant quietly rewriting your project out from under you. The draft-review stage protected you from a demo that looked done but was not. And this stage, the one you are finishing now, protects you from the quiet, compounding cost of a system that runs whether you are watching it or not.

None of these lessons expire. The next feature you add will need a job statement before it needs code. The next agent session will need boundaries before it needs a prompt. The next draft will need interrogation before it needs your trust. And every day this app stays live, it will need someone watching the meters. That someone is you.

You survived the build. Now you are equipped to survive the operation.

The meter is running. Keep your eyes open, watch the limits, and operate with confidence.

Back Matter

Ask the App What It Costs

You do not need to be an operations engineer to manage your costs. You just need to know how to ask the AI to do the audit for you.

Copy and paste these prompts into your builder any time you feel uncertain about your live architecture:

- **The Baseline Audit:** "List every service, database, or API this app depends on that could result in a bill after launch."
- **The Background Audit:** "List every background process, scheduled cron job, retry mechanism, webhook listener, or queue currently running in this codebase."
- **The AI Audit:** "List every place this app calls an AI model or paid API, and tell me if those calls have a strict rate limit."
- **The Storage Audit:** "List every place this app stores files, logs, generated content, or user data, and tell me exactly when that data is deleted."
- **The Abuse Audit:** "List every user action that could be intentionally abused to trigger repeated costs against my billing."
- **The Pre-Flight Check:** "List exactly what I should cap, alert, or disable before I add any new features."

The Safe Operation Checklist

Operating an app means managing a living system. Use this final protocol to ensure your system is under control before you walk away from the keyboard.

- ☐ **The Meters Are Visible:** I know exactly where this app spends money.
- ☐ **The Traps Are Set:** Hard billing limits are actively protecting my credit card.
- ☐ **The Background is Quiet:** I have audited every cron job, retry loop, and webhook.
- ☐ **The Edges Are Capped:** All AI inputs, file uploads, and API calls have strict maximum limits.
- ☐ **The Shutdown Plan Exists:** I know exactly how to pull the plug if things escalate.

The Complete Don’t Go Broke in the Age of AI Series

DON’T GO BROKE IN THE AGE OF AI — BOOKLET 1 OF 5

Before the Idea Becomes Reality

DON’T GO BROKE IN THE AGE OF AI — BOOKLET 2 OF 5

Before the Build Starts

DON’T GO BROKE IN THE AGE OF AI — BOOKLET 3 OF 5

Before the Agent Starts

DON’T GO BROKE IN THE AGE OF AI — BOOKLET 4 OF 5

Before You Trust the First Draft

DON’T GO BROKE IN THE AGE OF AI — BOOKLET 5 OF 5

After Launch, the Meter Still Runs